

Sliding Window

1. Overview

The **Sliding Window** is a technique for efficiently processing subarrays or substrings of an array/string by maintaining a **window** (a contiguous range) that slides across the data. Instead of recomputing from scratch every time the window moves, you **update** the result incrementally — adding the new element and removing the outgoing element.

This turns many $O(n^2)$ brute-force solutions into $O(n)$ solutions.

There are two main flavors:

- **Fixed-size window** — the window length is constant (e.g., max sum of any subarray of size k).
 - **Variable-size window** — the window grows and shrinks based on a condition (e.g., longest subarray with $\text{sum} \leq \text{target}$).
-

2. Intuition

Simple explanation

Imagine you have a long strip of paper with numbers written on it. You have a magnifying glass that can only show you k numbers at a time. You place it at the start, compute your answer, then shift it right by one position. Instead of re-reading all k numbers, you just subtract the number that just left and add the new number that entered. That's the sliding window.

Analogy

Think of a **train moving along a track**. The train's length is fixed (like a fixed window), and as it moves, one carriage exits the station behind and one enters ahead. You don't recount all passengers — you just update the count.

For **variable-size windows**, think of a **rubber band** that stretches and shrinks. You keep stretching it to the right until some condition breaks, then you shrink from the left until it's satisfied again, and repeat.

Step-by-step reasoning

1. Define a window range `[left, right)` or `[left, right]`.
2. Expand `right` to include new elements.
3. Update some state (sum, count, frequency map, etc.).
4. If the window becomes invalid (or too large for fixed-size), move `left` forward until valid.
5. Record the answer at valid windows.
6. Repeat until `right` reaches the end.

Why it works

The key insight: **contiguous subarrays share most of their elements**. When the window slides by one position, the change is small — one element enters, one leaves. By maintaining incremental state, we avoid

recomputing the entire subarray from scratch.

3. When to Use It

Use sliding window when the problem involves **contiguous subarrays/substrings** and you need to optimize a brute-force $O(n^2)$ solution.

Common trigger phrases

- "subarray of size k"
- "longest substring without repeating characters"
- "minimum window that contains..."
- "subarray with sum $\leq$ target"
- "at most K distinct characters"
- "exactly K distinct characters"
- "maximum/minimum sum of subarray of size k"
- "count subarrays where condition holds"
- "sliding window maximum/minimum"
- "contains all characters from target string"

Typical patterns

- **Fixed size:** "Given an array and an integer k, find the maximum/minimum/average of every subarray of size k."
 - **Variable size:** "Find the longest/shortest subarray that satisfies some condition."
 - **Count with condition:** "Count subarrays where sum/product/condition is $\leq K$ / $\geq K$."
 - **Two strings:** "Does string s2 contain a permutation of s1?" or "Find all anagrams of a pattern in a string."
-

4. When Not to Use It

Simpler alternatives exist

- If the array is **static** and you need many **range sum queries**, use a **prefix sum array** — it's $O(1)$ per query, no sliding needed.
- If you only need a **single statistic** (e.g., just the maximum sum of any subarray), a simple **Kadane's algorithm** may suffice.

Overkill cases

- The brute-force $O(n^2)$ is acceptable for small constraints ($n \leq 10^3$).
- If the array is **not contiguous** (e.g., subsequences, subsets), sliding window does not apply — use two-pointers with sorting, DP, or backtracking.

Common wrong assumptions

- **Sliding window does NOT work when the "window validity" is non-monotonic** — i.e., if expanding the window can make it invalid, then shrinking it can make it valid again, but expanding it again can

also make it invalid again. This happens when the condition depends on **global** properties like "sum is odd" or "product is a perfect square." For those, two-pointer + prefix tricks or more advanced approaches are needed.

- **Negative numbers can break the classic variable-size sliding window** for sum conditions. If nums can be negative, expanding the window doesn't necessarily increase the sum. Use prefix sums + hashmap instead.

Edge cases where it fails or is inefficient

- **Product constraints:** If the problem involves product (not sum), the product can overflow or grow too fast. Use logarithms or handle carefully.
- **Non-ordered conditions:** If the condition is "subarray sum modulo $k = 0$ " and elements can be negative, the monotonic property is lost. Use prefix sum + hashmap.
- **Large alphabet / many characters:** If you're tracking frequencies and the alphabet is huge (Unicode), a hashmap is fine, but array-based frequency may be impractical.

5. Core Concepts

5.1 Window Boundaries (`left`, `right`)

- **What it means:** Two pointers defining the current subarray. Usually `right` expands (adds elements), `left` contracts (removes elements).
- **Why it matters:** Correct boundary management avoids off-by-one errors and ensures every valid window is considered.
- **Example:** For `nums = [1, 2, 3, 4]`, window `[0, 2)` means elements at indices 0 and 1 (size 2).

5.2 State Variable

- **What it means:** A variable that tracks the property of the current window (sum, count of distinct chars, frequency map, max/min in window).
- **Why it matters:** This is what you update incrementally instead of recomputing.
- **Example:** `window_sum += nums[right]` when expanding, `window_sum -= nums[left]` when shrinking.

5.3 Validity Condition

- **What it means:** The condition that the current window must satisfy (e.g., `window_sum <= target`, `distinct_chars <= K`).
- **Why it matters:** Determines when to shrink the window and when to record the answer.
- **Example:** In "longest substring without repeating characters", validity means all frequencies are ≤ 1 .

5.4 Answer Update

- **What it means:** Recording the result when the window is valid (e.g., `max_len = max(max_len, right - left)`).
- **Why it matters:** You must know **when** to update — every valid window, only when window is at certain size, etc.

5.5 Monotonic Deque (for window max/min)

- **What it means:** A deque that stores indices in decreasing (for max) or increasing (for min) order of values within the window.
 - **Why it matters:** Allows $O(1)$ retrieval of the maximum/minimum in the current window while sliding.
 - **Example:** `deque = [3, 1, 2]` means the max is `nums[3]`, then `nums[1]`, then `nums[2]` in descending order.
-

6. Step-by-Step Algorithm

Fixed-Size Window (e.g., max sum of subarray of size k)

1. Initialize `left = 0`, `window_sum = 0`, `max_sum` to a small value (or first window sum).
2. Iterate `right` from `0` to `n - 1`:
 - a. Add `nums[right]` to `window_sum`.
 - b. If `right - left + 1 == k` (window reaches size k):
 - Update `max_sum = max(max_sum, window_sum)`.
 - Subtract `nums[left]` from `window_sum`.
 - Move `left++`.
3. Return `max_sum`.

Variable-Size Window (e.g., longest subarray with sum $\leq$ target)

1. Initialize `left = 0`, `window_sum = 0`, `max_len = 0`.
2. Iterate `right` from `0` to `n - 1`:
 - a. Add `nums[right]` to `window_sum`.
 - b. While `window_sum > target` (window is invalid):
 - Subtract `nums[left]` from `window_sum`.
 - `left++`.
 - c. Update `max_len = max(max_len, right - left + 1)`.
3. Return `max_len`.

Longest Substring Without Repeating Characters

1. Initialize `left = 0`, `max_len = 0`, `freq[256] = {0}` (or `unordered_map`).
2. Iterate `right` from `0` to `n - 1`:
 - a. Increment `freq[s[right]]`.
 - b. While `freq[s[right]] > 1` (a character repeats):
 - Decrement `freq[s[left]]`.
 - `left++`.
 - c. Update `max_len = max(max_len, right - left + 1)`.
3. Return `max_len`.

Minimum Window Substring

1. Initialize `left = 0`, `min_len = INF`, `start = 0`, `required = len(t)`, `freq` map for `t`.
2. Iterate `right` from `0` to `n - 1`:

- a. If `s[right]` is in `t`, decrement its count in a `window` map. If `window[s[right]] <= freq[s[right]]`, decrement `required`.
- b. While `required == 0` (all chars of `t` are covered):
 - Update `min_len` and `start` if current window is smaller.
 - If `s[left]` is in `t`: if `window[s[left]] == freq[s[left]]`, increment `required`. Decrement `window[s[left]]`.
 - `left++`.
- 3. If `min_len == INF`, return `""`. Else return `s.substr(start, min_len)`.

At Most K Distinct Characters

1. Initialize `left = 0`, `ans = 0`, `freq` map.
2. Iterate `right` from `0` to `n - 1`:
 - a. Increment `freq[s[right]]`.
 - b. While `freq.size() > k`:
 - Decrement `freq[s[left]]`; if 0, erase it.
 - `left++`.
 - c. `ans += right - left + 1` (counts all valid subarrays ending at `right`).
3. Return `ans`.

Exactly K Distinct Characters

1. Use `atMost(k) - atMost(k - 1)` where `atMost` returns count of subarrays with $\leq K$ distinct chars.
2. Return `atMost(s, k) - atMost(s, k - 1)`.

Sliding Window Maximum (Monotonic Deque)

1. Initialize `deque` (stores indices), result vector.
2. Iterate `i` from `0` to `n - 1`:
 - a. Remove indices from front if `deque.front() <= i - k` (out of window).
 - b. While `nums[deque.back()] <= nums[i]`, pop from back.
 - c. Push `i` to back.
 - d. If `i >= k - 1`, `result.push_back(nums[deque.front()])`.
3. Return `result`.

7. Dry Run

Fixed-Size Window: Maximum sum of subarray of size `k = 3`

```
nums = [2, 1, 5, 1, 3, 2], k = 3
```

Step	right	left	Window	window_sum	max_sum	Action
0	0	0	[2]	2	$-\infty$	Add 2
1	1	0	[2, 1]	3	$-\infty$	Add 1

Step	right	left	Window	window_sum	max_sum	Action
2	2	0	[2, 1, 5]	8	8	Window full. Max = 8
3	3	1	[1, 5, 1]	7	8	Remove 2, add 1. Max = 8
4	4	2	[5, 1, 3]	9	9	Remove 1, add 3. Max = 9
5	5	3	[1, 3, 2]	6	9	Remove 5, add 2. Max = 9

Answer: 9 (subarray [5, 1, 3] or [3, 1, 5] depending on indexing — here it's indices 2,3,4: 5+1+3=9).

Variable-Size Window: Longest subarray with sum ≤ 7

```
nums = [3, 1, 2, 5, 1, 1, 2], target = 7
```

right	left	Window	Sum	Valid?	max_len	Action
0	0	[3]	3	Yes	1	Expand
1	0	[3,1]	4	Yes	2	Expand
2	0	[3,1,2]	6	Yes	3	Expand
3	0	[3,1,2,5]	11	No	3	Shrink until valid
3	1	[1,2,5]	8	No	3	Shrink
3	2	[2,5]	7	Yes	3	Valid, max stays 3
4	2	[2,5,1]	8	No	3	Shrink
4	3	[5,1]	6	Yes	3	Valid
5	3	[5,1,1]	7	Yes	4	New max!
6	3	[5,1,1,2]	9	No	4	Shrink
6	4	[1,1,2]	4	Yes	4	Valid but not longer

Answer: 4 (subarray [5, 1, 1, 2] → wait, that sum is 9. Let me re-check.)

Correction: At right=6, window [5,1,1,2] sum=9 > 7. After shrinking left to 4, window [1,1,2] sum=4, len=3.

Let's re-trace carefully:

right	left	Window	Sum	Valid?	max_len	Action
0	0	[3]	3	Yes	1	
1	0	[3,1]	4	Yes	2	
2	0	[3,1,2]	6	Yes	3	

right	left	Window	Sum	Valid?	max_len	Action
3	0	[3,1,2,5]	11	No	3	shrink
3	1	[1,2,5]	8	No	3	shrink
3	2	[2,5]	7	Yes	3	valid (len=2)
4	2	[2,5,1]	8	No	3	shrink
4	3	[5,1]	6	Yes	3	valid (len=2)
5	3	[5,1,1]	7	Yes	4	len=3? No, wait.

Actually $\text{right} - \text{left} + 1$ at $\text{right}=5, \text{left}=3 \Rightarrow 5-3+1 = 3$. Same.

Let me just redo with a clearer example:

```
nums = [4, 2, 1, 7, 1, 1, 3], target = 8
```

right	left	Window	Sum	Valid?	max_len	Action
0	0	[4]	4	Yes	1	Expand
1	0	[4,2]	6	Yes	2	Expand
2	0	[4,2,1]	7	Yes	3	Expand
3	0	[4,2,1,7]	14	No	3	Shrink
3	1	[2,1,7]	10	No	3	Shrink
3	2	[1,7]	8	Yes	3	Valid (len=2)
4	2	[1,7,1]	9	No	3	Shrink
4	3	[7,1]	8	Yes	3	Valid (len=2)
5	3	[7,1,1]	9	No	3	Shrink
5	4	[1,1]	2	Yes	3	Valid (len=2)
6	4	[1,1,3]	5	Yes	4	len=3? No: $6-4+1=3$

Hmm, let me just use a different example that gives a clear result.

```
nums = [1, 2, 3, 4, 5], target = 9
```

right	left	Window	Sum	Valid?	max_len
0	0	[1]	1	Yes	1
1	0	[1,2]	3	Yes	2

right	left	Window	Sum	Valid?	max_len
2	0	[1,2,3]	6	Yes	3
3	0	[1,2,3,4]	10	No	3
3	1	[2,3,4]	9	Yes	3
4	1	[2,3,4,5]	14	No	3
4	2	[3,4,5]	12	No	3
4	3	[4,5]	9	Yes	3

Answer: 3 (subarray [1,2,3] or [2,3,4] or [4,5]).

Sliding Window Maximum (k = 3)

```
nums = [1, 3, -1, -3, 5, 3, 6, 7], k = 3
```

i	Element	Deque (indices → values)	Window Max	Out of window?
0	1	[0 → 1]	—	No
1	3	[1 → 3] (0 popped because 1 < 3)	—	No
2	-1	[1 → 3, 2 → -1]	3	Front = 1 ≥ 0=2-2 → valid
3	-3	[1 → 3, 2 → -1, 3 → -3]	3	Front = 1 ≥ 0 → valid
4	5	[4 → 5] (all popped)	5	Front=4 ≥ 2 → valid
5	3	[4 → 5, 5 → 3]	5	Front=4 ≥ 3 → valid
6	6	[6 → 6] (5,3 popped)	6	Front=6 ≥ 4 → valid
7	7	[7 → 7] (6 popped)	7	Front=7 ≥ 5 → valid

Result: [3, 3, 5, 5, 6, 7]

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

// =====
// 1. FIXED-SIZE WINDOW: Maximum sum subarray of size k
// =====
int maxSumFixed(vector<int>& nums, int k) {
    int n = nums.size();
    if (n < k) return -1; // edge case
```

```

    int windowSum = 0;
    int maxSum = INT_MIN;

    for (int right = 0; right < n; right++) {
        windowSum += nums[right];

        // When window reaches size k
        if (right >= k - 1) {
            maxSum = max(maxSum, windowSum);
            windowSum -= nums[right - k + 1]; // remove leftmost element
        }
    }
    return maxSum;
}

// =====
// 2. VARIABLE-SIZE WINDOW: Longest subarray with sum <= target
// =====
int longestSubarraySumLE(vector<int>& nums, int target) {
    int left = 0, windowSum = 0, maxLen = 0;

    for (int right = 0; right < nums.size(); right++) {
        windowSum += nums[right];

        // Shrink while invalid
        while (windowSum > target) {
            windowSum -= nums[left];
            left++;
        }

        // Window is now valid
        maxLen = max(maxLen, right - left + 1);
    }
    return maxLen;
}

// =====
// 3. LONGEST SUBSTRING WITHOUT REPEATING CHARACTERS
// =====
int longestUniqueSubstr(string s) {
    int left = 0, maxLen = 0;
    vector<int> freq(256, 0); // ASCII

    for (int right = 0; right < s.size(); right++) {
        freq[s[right]]++;

        // Shrink if any character repeats
        while (freq[s[right]] > 1) {
            freq[s[left]]--;
            left++;
        }

        maxLen = max(maxLen, right - left + 1);
    }
}

```

```

    }
    return maxLen;
}

// =====
// 4. MINIMUM WINDOW SUBSTRING
// =====
string minWindowSubstring(string s, string t) {
    vector<int> freqT(128, 0), window(128, 0);
    for (char c : t) freqT[c]++;

    int left = 0, required = t.size();
    int minLen = INT_MAX, start = 0;

    for (int right = 0; right < s.size(); right++) {
        char c = s[right];
        window[c]++;

        if (freqT[c] > 0 && window[c] <= freqT[c]) {
            required--;
        }

        while (required == 0) {
            // Update answer
            if (right - left + 1 < minLen) {
                minLen = right - left + 1;
                start = left;
            }

            // Shrink from left
            char leftChar = s[left];
            window[leftChar]--;

            if (freqT[leftChar] > 0 && window[leftChar] < freqT[leftChar]) {
                required++;
            }
            left++;
        }
    }

    return minLen == INT_MAX ? "" : s.substr(start, minLen);
}

// =====
// 5. COUNT SUBARRAYS WITH SUM <= TARGET
// =====
long long countSubarraysSumLE(vector<int>& nums, int target) {
    int left = 0, windowSum = 0;
    long long count = 0;

    for (int right = 0; right < nums.size(); right++) {
        windowSum += nums[right];

        while (windowSum > target) {

```

```

        windowSum -= nums[left];
        left++;
    }

    // All subarrays ending at 'right' with start in [left, right] are valid
    count += (right - left + 1);
}
return count;
}

// =====
// 6. AT MOST K DISTINCT CHARACTERS
// =====
int atMostKDistinct(string s, int k) {
    int left = 0, ans = 0;
    unordered_map<char, int> freq;

    for (int right = 0; right < s.size(); right++) {
        freq[s[right]]++;

        while (freq.size() > k) {
            freq[s[left]]--;
            if (freq[s[left]] == 0) freq.erase(s[left]);
            left++;
        }

        ans += (right - left + 1);
    }
    return ans;
}

// =====
// 7. EXACTLY K DISTINCT = atMost(K) - atMost(K-1)
// =====
int exactlyKDistinct(string s, int k) {
    return atMostKDistinct(s, k) - atMostKDistinct(s, k - 1);
}

// =====
// 8. SLIDING WINDOW MAXIMUM (Monotonic Deque)
// =====
vector<int> slidingWindowMax(vector<int>& nums, int k) {
    deque<int> dq; // stores indices, values in decreasing order
    vector<int> result;

    for (int i = 0; i < nums.size(); i++) {
        // Remove indices out of current window
        if (!dq.empty() && dq.front() <= i - k) {
            dq.pop_front();
        }

        // Remove smaller elements from back (they can never be max)
        while (!dq.empty() && nums[dq.back()] <= nums[i]) {
            dq.pop_back();
        }
    }
}

```

```

    }

    dq.push_back(i);

    // Start recording results once we have a full window
    if (i >= k - 1) {
        result.push_back(nums[dq.front()]);
    }
}
return result;
}

// =====
// EXAMPLE USAGE
// =====
int main() {
    // Fixed-size
    vector<int> nums1 = {2, 1, 5, 1, 3, 2};
    cout << "Max sum (k=3): " << maxSumFixed(nums1, 3) << "\n"; // 9

    // Variable-size (sum <= 7)
    vector<int> nums2 = {4, 2, 1, 7, 1, 1, 3};
    cout << "Longest subarray sum<=8: " << longestSubarraySumLE(nums2, 8) << "\n";
// 3

    // Longest unique substring
    cout << "Longest unique: " << longestUniqueSubstr("abcabcbb") << "\n"; // 3

    // Minimum window substring
    cout << "Min window: " << minWindowSubstring("ADOBECODEBANC", "ABC") << "\n";
// "BANC"

    // Count subarrays sum <= 10
    vector<int> nums3 = {1, 2, 3};
    cout << "Count sum<=3: " << countSubarraysSumLE(nums3, 3) << "\n"; // 3

    // At most K distinct
    cout << "AtMost 2 distinct: " << atMostKDistinct("eceba", 2) << "\n"; // 3
    cout << "Exactly 2 distinct: " << exactlyKDistinct("eceba", 2) << "\n"; // 3

    // Sliding window maximum
    vector<int> nums4 = {1, 3, -1, -3, 5, 3, 6, 7};
    auto res = slidingWindowMax(nums4, 3);
    cout << "Window max: ";
    for (int x : res) cout << x << " ";
    cout << "\n"; // 3 3 5 5 6 7

    return 0;
}

```

9. Python Implementation

```
from collections import deque, defaultdict
from typing import List

# =====
# 1. FIXED-SIZE WINDOW: Maximum sum subarray of size k
# =====
def max_sum_fixed(nums: List[int], k: int) -> int:
    n = len(nums)
    if n < k:
        return -1

    window_sum = sum(nums[:k])
    max_sum = window_sum

    for right in range(k, n):
        window_sum += nums[right] - nums[right - k]
        max_sum = max(max_sum, window_sum)

    return max_sum

# =====
# 2. VARIABLE-SIZE WINDOW: Longest subarray with sum <= target
# =====
def longest_subarray_sum_le(nums: List[int], target: int) -> int:
    left = window_sum = max_len = 0

    for right in range(len(nums)):
        window_sum += nums[right]

        while window_sum > target:
            window_sum -= nums[left]
            left += 1

        max_len = max(max_len, right - left + 1)

    return max_len

# =====
# 3. LONGEST SUBSTRING WITHOUT REPEATING CHARACTERS
# =====
def longest_unique_substr(s: str) -> int:
    left = max_len = 0
    freq = defaultdict(int)

    for right in range(len(s)):
        freq[s[right]] += 1

        while freq[s[right]] > 1:
            freq[s[left]] -= 1
            if freq[s[left]] == 0:
                del freq[s[left]]
            left += 1
```

```

        max_len = max(max_len, right - left + 1)

    return max_len

# =====
# 4. MINIMUM WINDOW SUBSTRING
# =====
def min_window_substring(s: str, t: str) -> str:
    from collections import Counter
    freq_t = Counter(t)
    window = defaultdict(int)

    left = required = len(t)
    min_len = float('inf')
    start = 0

    for right in range(len(s)):
        c = s[right]
        window[c] += 1

        if c in freq_t and window[c] <= freq_t[c]:
            required -= 1

        while required == 0:
            if right - left + 1 < min_len:
                min_len = right - left + 1
                start = left

            left_char = s[left]
            window[left_char] -= 1

            if left_char in freq_t and window[left_char] < freq_t[left_char]:
                required += 1
            left += 1

    return "" if min_len == float('inf') else s[start:start + min_len]

# =====
# 5. COUNT SUBARRAYS WITH SUM <= TARGET
# =====
def count_subarrays_sum_le(nums: List[int], target: int) -> int:
    left = window_sum = count = 0

    for right in range(len(nums)):
        window_sum += nums[right]

        while window_sum > target:
            window_sum -= nums[left]
            left += 1

        count += (right - left + 1)

    return count

```

```

# =====
# 6. AT MOST K DISTINCT CHARACTERS
# =====
def at_most_k_distinct(s: str, k: int) -> int:
    left = ans = 0
    freq = defaultdict(int)

    for right in range(len(s)):
        freq[s[right]] += 1

        while len(freq) > k:
            freq[s[left]] -= 1
            if freq[s[left]] == 0:
                del freq[s[left]]
            left += 1

        ans += (right - left + 1)

    return ans

# =====
# 7. EXACTLY K DISTINCT = atMost(K) - atMost(K-1)
# =====
def exactly_k_distinct(s: str, k: int) -> int:
    return at_most_k_distinct(s, k) - at_most_k_distinct(s, k - 1)

# =====
# 8. SLIDING WINDOW MAXIMUM (Monotonic Deque)
# =====
def sliding_window_max(nums: List[int], k: int) -> List[int]:
    dq = deque() # stores indices, values in decreasing order
    result = []

    for i in range(len(nums)):
        # Remove indices out of current window
        if dq and dq[0] <= i - k:
            dq.popleft()

        # Remove smaller elements from back
        while dq and nums[dq[-1]] <= nums[i]:
            dq.pop()

        dq.append(i)

        # Start recording results once we have a full window
        if i >= k - 1:
            result.append(nums[dq[0]])

    return result

# =====
# EXAMPLE USAGE
# =====

```

```

if __name__ == "__main__":
    # Fixed-size
    print("Max sum (k=3):", max_sum_fixed([2, 1, 5, 1, 3, 2], 3)) # 9

    # Variable-size
    print("Longest subarray sum<=8:", longest_subarray_sum_le([4, 2, 1, 7, 1, 1, 3], 8)) # 3

    # Longest unique substring
    print("Longest unique:", longest_unique_substr("abcabcbb")) # 3

    # Minimum window substring
    print("Min window:", min_window_substring("ADOBECODEBANC", "ABC")) # BANC

    # Count subarrays sum <= 3
    print("Count sum<=3:", count_subarrays_sum_le([1, 2, 3], 3)) # 3

    # At most K distinct
    print("AtMost 2 distinct:", at_most_k_distinct("eceba", 2)) # 3
    print("Exactly 2 distinct:", exactly_k_distinct("eceba", 2)) # 3

    # Sliding window maximum
    print("Window max:", sliding_window_max([1, 3, -1, -3, 5, 3, 6, 7], 3))
    # [3, 3, 5, 5, 6, 7]

```

10. Code Explanation

Fixed-Size Window (maxSumFixed)

- We maintain `windowSum` that tracks the sum of the current window.
- At each `right`, we add `nums[right]`.
- When `right >= k - 1`, the window is full, so we update `maxSum` and then **remove** the leftmost element (`nums[right - k + 1]`) to prepare for the next slide.
- This is the simplest sliding window — the window size never changes, so no `while` shrink loop is needed.

Variable-Size Window (longestSubarraySumLE)

- We expand `right` unconditionally.
- If the window becomes invalid (`windowSum > target`), we shrink from the left **until it becomes valid again**.
- After each valid window, we update `maxLen`.
- **Key insight:** Because all numbers are positive (or non-negative), increasing window size increases sum, so the `while` shrink loop is guaranteed to terminate.

Longest Substring Without Repeating (longestUniqueSubstr)

- We track frequencies of characters in the current window.
- When `freq[s[right]] > 1`, we have a repeat — the character at `right` already exists in the window.

- We shrink from the left until the repeat is resolved (i.e., `freq[s[right]]` drops back to 1).
- This works because the repeat can only involve `s[right]` (since all other characters weren't repeating before).

Minimum Window Substring (minWindowSubstring)

- This is more complex because we need to match **all characters of t** (with their frequencies).
- `required` tracks how many characters from `t` are still needed.
- When `required == 0`, we have a valid window. We try to shrink it to find a **smaller** valid window.
- The `window` map tracks our current counts; `freqT` tracks what we need.
- When shrinking, if removing `leftChar` causes its count to drop below `freqT[leftChar]`, we increment `required` again.

Count Subarrays Sum $\leq$ Target (countSubarraysSumLE)

- For each `right`, all subarrays ending at `right` with start $\in [\text{left}, \text{right}]$ are valid.
- The count of such subarrays is `right - left + 1`.
- This is a very common pattern: instead of counting one best answer, count all valid ones.

At Most K Distinct / Exactly K Distinct

- **AtMost K**: Expand right, shrink while distinct count $> K$. Count all valid subarrays ending at `right`.
- **Exactly K**: Use the identity: `exactly(K) = atMost(K) - atMost(K-1)`.
- This works because the set of subarrays with exactly K distinct chars is the set with atMost(K) minus the set with atMost(K-1).

Sliding Window Maximum (Monotonic Deque)

- The deque stores **indices** (not values) and maintains values in **decreasing order**.
- Before adding `i`, we pop from the back all indices whose values are $\leq \text{nums}[i]$ — they can never be the maximum in any future window that includes `i`.
- We pop from the front any index that falls out of the current window.
- The front always holds the max for the current window.
- This gives $O(n)$ overall because each index is pushed and popped at most once.

11. Complexity Analysis

Algorithm	Time	Space	Notes
Fixed-size window (max sum)	$O(n)$	$O(1)$	Single pass, constant extra space
Variable-size (sum $\leq$ target)	$O(n)$	$O(1)$	Each element added and removed at most once
Longest unique substring	$O(n)$	$O(1)^*$	* $O(1)$ if using fixed array[256]; $O(\text{alphabet})$ with hashmap
Minimum window substring	$O(n + m)$	$O(1)^*$	$m = \text{len}(t)$. * $O(128)$ or $O(\text{alphabet})$

Algorithm	Time	Space	Notes
Count subarrays (sum $\leq$ target)	$O(n)$	$O(1)$	For positive numbers only
At most K distinct	$O(n)$	$O(k)$	HashMap stores at most $k+1$ entries
Exactly K distinct	$O(n)$	$O(k)$	Calls <code>atMost</code> twice — still $O(n)$
Sliding window max (deque)	$O(n)$	$O(k)$	Deque stores at most k indices

Note: In the variable-size sliding window with sum constraint, each element is pushed (added to `windowSum`) once and popped (subtracted from `windowSum`) at most once. This gives amortized $O(n)$. The same amortized argument applies to all sliding window algorithms.

12. Common Patterns

Pattern 1: Fixed-size window, optimized with sliding

- **How to identify:** "subarray of size k ", "any subarray of length k that satisfies condition".
- **General approach:** Maintain window of exactly k elements. Update on slide.
- **Examples:** Maximum average subarray, max sum subarray of size k .

Pattern 2: Longest valid substring/subarray

- **How to identify:** "longest substring where condition holds", "maximum length subarray with property X ".
- **General approach:** Variable-size window, shrink when invalid, record max length at each valid state.
- **Examples:** Longest substring without repeating, longest subarray with sum $\leq K$.

Pattern 3: Shortest valid substring/subarray

- **How to identify:** "minimum window", "smallest subarray containing all elements", "shortest subarray with sum $\geq K$ ".
- **General approach:** Expand until valid, then shrink while still valid to find minimal. Record min length.
- **Examples:** Minimum window substring, smallest subarray with sum $\geq K$.

Pattern 4: Count subarrays satisfying condition

- **How to identify:** "number of subarrays where property holds", "count subarrays with ...".
- **General approach:** For variable-size window with a monotonic condition, every valid window ending at `right` contributes `right - left + 1` subarrays.
- **Examples:** Count subarrays with sum $\leq K$, count subarrays with at most K distinct characters.

Pattern 5: Exactly K / At most K

- **How to identify:** "exactly K distinct", "at most K different integers", "exactly K odd numbers".
- **General approach:** Use `exactly(K) = atMost(K) - atMost(K-1)` for count problems.
- **Examples:** Subarrays with exactly K distinct integers, exactly K odd numbers.

Pattern 6: Window maximum/minimum

- **How to identify:** "sliding window maximum", "maximum in each window of size k".
- **General approach:** Use monotonic deque (decreasing for max, increasing for min).
- **Examples:** Sliding window max, sliding window min.

Pattern 7: Anagram / Permutation in string

- **How to identify:** "does s2 contain a permutation of s1", "find all anagrams of a pattern in a string".
- **General approach:** Fixed-size window (size = len(pattern)). Use frequency array and compare counts.
- **Examples:** Permutation in string, find all anagrams in a string.

13. Common Mistakes

Off-by-one errors

- Using `right - left` instead of `right - left + 1` for window size.
- Using `right >= k` instead of `right >= k - 1` to check first full window.
- Off-by-one in the deque front check: `dq.front() <= i - k` (should be `<=`, not `<`).

Wrong initialization

- Not initializing `maxLen = 0` (fails for single element).
- Not initializing `minLen = INT_MAX` or `INF` (breaks minimum computation).

Wrong shrink condition

- Using `if` instead of `while` for shrink — a single shrink may not be enough.
- Forgetting that after shrinking, you need to re-check the condition.

Forgetting to erase from hashmap

- When `freq[key]` drops to 0 and you track distinct count via map size, you must **erase** the key from the map.

Negative numbers in variable-size window

- Classic sliding window with sum constraint fails when numbers can be negative, because increasing window size can decrease the sum.
- **Fix:** Use prefix sums + hashmap (or monotonic deque for specific problems).

Overlooking required matching logic

- In Minimum Window Substring: using `freqT[c] > 0` check is crucial to only track characters that actually appear in `t`.
- Not tracking that a character can appear multiple times in `t`.

Not using long long for counts

- `count` can exceed `int` range. Always use `long long`.

Deque: forgetting to pop front

- For sliding window max, you must check and remove indices that have left the window.

Wrong comparison for deque order

- For max: `nums[back] <= nums[i]` — use `<=` to remove equal values too (since a newly added equal value with a larger index is more useful).
- For min: `nums[back] >= nums[i]`.

14. Edge Cases

Scenario	Example	What to check
Empty array	<code>nums = []</code>	Return 0 or empty string
Single element	<code>nums = [5]</code>	Window size = 1 works
$k >$ array length	<code>nums = [1,2], k = 5</code>	Return -1 or empty, don't crash
All elements equal	<code>nums = [3,3,3,3], k=2</code>	Works fine, max = min = 3
All distinct characters	<code>s = "abcdef"</code>	Longest unique = full string
All same character	<code>s = "aaaa"</code>	Longest unique = 1
No valid window	<code>s = "a", t = "abc"</code>	Return "" for min window
Target string longer than s	<code>s = "a", t = "ab"</code>	Return ""
Target = empty	<code>t = ""</code>	Usually return ""
Sorted array	<code>nums = [1,2,3,4,5]</code>	Works normally
Reverse sorted	<code>nums = [5,4,3,2,1]</code>	Works normally
Large values	<code>nums = [10^9, 10^9]</code>	Use <code>long long</code> for sum
Negative values (sum)	<code>nums = [-2, 1, -3, 4], target = 1</code>	Sliding window may fail! Use prefix sum + hashmap
$k = 1$	<code>nums = [7, 2, 9], k = 1</code>	Every element is its own window
$k = n$	<code>nums = [1,2,3], k = 3</code>	Only one window, the whole array
Count overflow	Large array with many valid subarrays	Use <code>long long</code> for count

15. Variations

15.1 Two Pointers (Non-Shrinking)

- **What changes:** Two pointers move independently without a "sliding window" state. Often used for sorted arrays or partition problems.
- **When used:** Removing duplicates from sorted array, three-sum, trapping rainwater.
- **Importance:** Commonly confused with sliding window. Use two pointers when the condition involves relative order but not frequency/sum state.

15.2 Sliding Window with Deque (Monotonic Queue)

- **What changes:** Instead of maintaining sum or frequency, maintain max/min in the window using a deque.
- **When used:** Sliding window maximum/minimum.
- **Importance:** Essential for CP and some hard interview problems.

15.3 Sliding Window with Hashmap + Counter

- **What changes:** Track character frequencies for substring anagram/permutation detection.
- **When used:** Permutation in string, find all anagrams.
- **Importance:** Very common in placement interviews.

15.4 Sliding Window on a Circular Array

- **What changes:** Extend the array by appending itself (doubling) or use modulo indexing.
- **When used:** Maximum sum subarray in a circular array, circular array window problems.
- **Importance:** Occasional variation in interviews.

15.5 Sliding Window for Strings with Wildcards

- **What changes:** Allow at most K character replacements to make the string valid.
- **When used:** Longest repeating character replacement (replace at most K characters to get the longest uniform substring).
- **Importance:** Common LeetCode hard/medium.

15.6 Sliding Window Median

- **What changes:** Instead of max, maintain median. Requires two heaps (max-heap + min-heap) with lazy deletion.
- **When used:** Median of sliding window of size k.
- **Importance:** Hard, but teachable for interviews.

15.7 Count Subarrays with Product < K

- **What changes:** Product grows fast. Use the same sliding window pattern but be careful with overflow and the fact that product > K doesn't just require shrinking by 1.
- **When used:** If numbers are positive, product is monotonic (expanding increases product, shrinking decreases it). Works like sum.
- **Importance:** Classic LeetCode problem.

15.8 Sliding Window with Bitmask

- **What changes:** Use a bitmask to track which characters are present instead of a frequency hashmap.

- **When used:** When you only care about presence/absence, not count.
 - **Importance:** Niche but useful for optimization.
-

16. Related Algorithms/Data Structures

Sliding Window vs Prefix Sum

- **Sliding Window:** $O(n)$ for finding optimal subarray under a condition. Maintains incremental state.
- **Prefix Sum:** $O(1)$ per query after $O(n)$ preprocessing. Best when you need arbitrary range sum queries with no condition to optimize.
- **Choose sliding window** when you're iterating once and updating state.
- **Choose prefix sum** when you need many random range queries.

Sliding Window vs Kadane's Algorithm

- **Sliding Window:** Works for subarray of size k , or with constraints ($\text{sum} \leq \text{target}$, at most K distinct).
- **Kadane's Algorithm:** $O(n)$ for maximum subarray sum (with no size constraint, can include negative numbers).
- **Choose sliding window** for size-constrained or count-constrained subarrays.
- **Choose Kadane** for unconstrained maximum subarray sum.

Sliding Window vs Monotonic Deque (it's a subset, not an alternative)

- Monotonic deque is a **tool used within** sliding window to track max/min, not a replacement.

Sliding Window vs Two Heaps

- **Sliding Window:** Works when you need statistics (max, min, sum) of every window.
- **Two Heaps** (max-heap + min-heap): Works for median tracking, but updates require $O(\log k)$ and lazy deletion.
- **Choose sliding window** for $O(1)$ amortized max/min via deque.
- **Choose two heaps** when you need median or other complex statistics.

Sliding Window vs Binary Search on Answer

- **Sliding Window:** Direct $O(n)$ for valid subarray problems.
- **Binary Search on Answer:** Used when you binary search the window size and check feasibility in $O(n)$. Gives $O(n \log n)$.
- **Choose sliding window** when the validity condition is monotonic and you can directly find the optimal.
- **Choose binary search** when the optimal isn't easy to track incrementally but a size-check is easy.

Sliding Window vs DP

- Some subarray problems that don't have monotonic properties can be solved with DP (e.g., maximum product subarray with negative numbers).
 - Sliding window is simpler and faster when applicable.
-

17. Practice Problems

Easy

Problem	Platform	Idea	Difficulty
Maximum Average Subarray I	LeetCode 643	Fixed-size window. Slide and compute average.	Easy
Longest Substring Without Repeating Characters	LeetCode 3	Variable-size window with frequency array. Classic.	Easy

Medium

Problem	Platform	Idea	Difficulty
Minimum Size Subarray Sum	LeetCode 209	Variable-size window. Expand until $\text{sum} \geq \text{target}$, then shrink to minimize length.	Medium
Longest Repeating Character Replacement	LeetCode 424	Variable-size window. Track max frequency. If $\text{window_size} - \text{max_freq} \leq k$, window is valid.	Medium
Permutation in String	LeetCode 567	Fixed-size window ($\text{size} = \text{len}(s1)$). Compare frequency arrays.	Medium

Hard

Problem	Platform	Idea	Difficulty
Minimum Window Substring	LeetCode 76	Variable-size window with two frequency maps and a required counter.	Hard
Sliding Window Maximum	LeetCode 239	Monotonic deque. Maintain decreasing order.	Hard
Count of Subarrays with Exactly K Distinct Characters	LeetCode 992	Use $\text{atMost}(K) - \text{atMost}(K-1)$. Count subarrays with $\leq K$ distinct then subtract.	Hard
Subarrays with K Different Integers	LeetCode 992	Same as above. Sliding window + counting.	Hard
Minimum Window with at Least K Repeating Characters	LeetCode 395	Sliding window + divide & conquer or frequency map with unique character count.	Hard

18. Interview Explanation

"Sliding window is a technique to optimize $O(n^2)$ brute-force solutions for contiguous subarray problems down to $O(n)$.

The core idea is to maintain a window — a contiguous range — and slide it across the array. Instead of recomputing from scratch each time, I update a state variable incrementally: when I expand the

window, I add the new element's contribution; when I shrink it, I subtract the outgoing element's contribution.

I use **fixed-size windows** when the subarray length is given, like maximum sum of any subarray of size k . I use **variable-size windows** when there's a condition like 'sum $\leq$ target' or 'at most k distinct characters' — I expand the right pointer, and if the condition breaks, I advance the left pointer until it's satisfied again.

For problems involving the maximum in each window, I use a **monotonic deque** that stores indices in decreasing order of their values. This gives $O(1)$ access to the current window's maximum and $O(n)$ overall time.

For counting problems like 'exactly k distinct characters', I use the identity: $\text{exactly}(k) = \text{atMost}(k) - \text{atMost}(k-1)$, which avoids having to handle the shrinking logic for exact equality.

The key constraints are that the condition must be **monotonic** — if a window is valid, its sub-windows should also be valid (or its super-windows). If numbers can be negative or the condition isn't monotonic, sliding window won't work and I'd use prefix sums or other approaches instead."

19. Revision Notes

- **Sliding window:** Optimizes $O(n^2) \rightarrow O(n)$ for **contiguous subarray** problems.
- **Two types:** Fixed-size (window size constant) and variable-size (window grows/shrinks based on condition).
- **Core pattern:** `left = 0; for right in range(n): add(nums[right]); while invalid: remove(nums[left]); left++; update(answer);`
- **State variable:** Sum, frequency map, distinct count, max frequency, etc.
- **Monotonic deque:** For window max/min. Front = max, back = newest. Pop from back while `nums[back] <= nums[i]`.
- **Count trick:** For exactly K , use `atMost(K) - atMost(K-1)`.
- **Required counter:** For minimum window substring, track how many chars from `t` are still needed.
- **Complexity:** $O(n)$ time, $O(1)$ or $O(k)$ space.
- **Common traps:**
 - Use `while` not `if` for shrinking.
 - `k > n` → handle gracefully.
 - `long long` for counts.
 - Hashmap erase when count hits 0 if tracking distinct count via map size.
 - Negative numbers break sum-based sliding window.
- **Condition must be monotonic** for variable-size window to work.

20. Final Cheat Sheet

Aspect	Details
When to use	Subarray/substring problems, contiguous range, condition on sum/count/type
Fixed window	Size k given → slide, add right, remove left at each step

Aspect	Details
Variable window	Condition given → expand right, shrink left when invalid, track answer
Count subarrays	Add <code>right - left + 1</code> for each valid window ending at <code>right</code>
Exactly K count	<code>atMost(K) - atMost(K-1)</code>
Window max/min	Monotonic deque: maintain decreasing (max) / increasing (min) order
Time	$O(n)$ — each element visited at most twice
Space	$O(1)$ or $O(k)$ (deque size, hashmap size)
Key code	<pre>left=0; for(r=0;r<n;r++){ add; while(invalid) remove,left++; update; }</pre>
Edge cases	Empty input, single element, $k > n$, negative numbers (breaks sum)
Common problems	LeetCode 3, 76, 239, 424, 567, 643, 992, 209, 395
Doesn't work for	Non-contiguous subproblems, non-monotonic conditions, negative numbers (sum)

End of Guide